



CP353201

Software Quality Assurance

SQA Defects4J Benchmark

การเปรียบเทียบการสร้าง ชุดทดสอบหน่วยภาษา Java

ด้วย Hill Climbing, AVM, GPT และ Gemini

จัดทำโดย

กรมกัญ พิริยะ • วัชรพล ดวงทองเงิน • ศิระพัทธ์ วงศ์วิวัฒน์เสรี

โจทย์ของการทดลอง

เปรียบเทียบความสามารถในการสร้างชุดทดสอบที่ใช้งานได้ ตรวจบักได้ และมี coverage

854

เคสจาก Defects4j

17

โปรเจกต์ Java

4

วิธีที่เปรียบเทียบ

3,416

งานทดลองทั้งหมด

หลัก

- แต่ละวิธีสร้างชุดทดสอบที่ใช้งานได้เพียงใด
- ตรวจพบบักได้ที่เคส และครอบคลุมโค้ดได้มากแค่ไหน
- ใช้เวลาเท่าใด และข้อจำกัดของแต่ละวิธีอยู่ตรงไหน

ขอบเขตสำคัญ: ผลนี้เป็นผลเชิงพรรณนาของ implementation + prompt + configuration ที่ใช้จริง

ขั้นตอนการประเมิน

สร้างชุดทดสอบจาก fixed revision แล้วนำชุดเดียวกันไปตรวจบน buggy revision

1 Fixed revision

ใช้สร้าง input และ expected behavior โดยไม่เห็น buggy source/patch

2 Generate JUnit tests

สร้างชุดทดสอบจากวิธีที่กำหนด แล้ว compile เป็น JUnit 4

3 Pass on fixed = Valid

ถ้าคอมไพล์และผ่านบน fixed จึงนับว่าใช้งานได้

4 Fail on buggy = Detect

รัน test เดิมบน buggy revision ถ้าล้มเหลวจึงนับว่าตรวจบักได้

เกณฑ์การนับผล

Completed ≠ Valid

งานที่รันจบยังไม่ถือว่าใช้ได้ จนกว่าจะ compile และผ่านบน fixed revision

Valid แต่ไม่มี coverage = Error

กรณี test ผ่าน แต่เก็บ coverage ไม่สำเร็จ จะไม่นำไปสรุป coverage

Detect bug

ต้องเป็นชุดทดสอบเดียวกันที่ผ่านบน fixed แต่ล้มเหลวบน buggy revision

สรุป: ใช้ fixed revision เพื่อสร้าง oracle แล้ววัดว่าชุดทดสอบเดียวกันทำให้ buggy revision ล้มเหลวได้หรือไม่

วิธีที่นำมาเปรียบเทียบ

แยกเป็นสองกลุ่ม: search-based algorithms และ AI-based tools

Search-based

Hill Climbing

ปรับ candidate ทีละขั้น
และสุ่ม restart เมื่อคะแนน
ไม่ดีขึ้น

AVM

ปรับตัวแปรเป็นลำดับ
ทดลองค่าข้างเคียงและ
pattern moves

ลักษณะร่วม

- $\text{Fitness} = \text{branch_ratio} + 0.001 \times \text{instruction_ratio}$ ของคลาสเป้าหมาย
- Oracle = รับบน fixed revision แล้วสร้าง assertion จากพฤติกรรมที่สังเกตได้
- จุดแข็ง: ควบคุมขอบเขต input ได้เป็นระบบ
- ข้อจำกัด: ติด construction/search domain ทำให้บางเคส unsupported

AI-based

GPT 5.6-terra

สร้างชุดทดสอบจาก fixed
source/API และ prompt
เฉพาะโมเดล

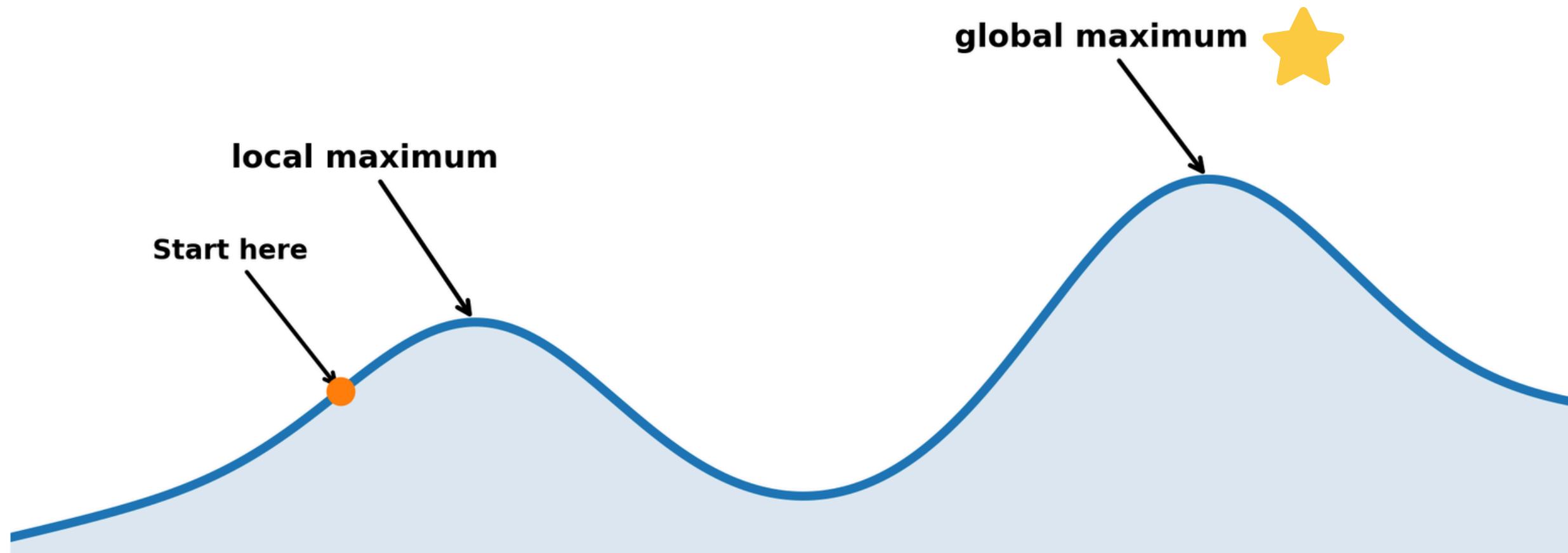
Gemini 3.5-flash-lite

สร้างชุดทดสอบจาก fixed
context และผ่าน
evaluator เดียวกัน

ลักษณะร่วม

- ยึดอยู่กับ input, setup และ assertion มากกว่า search domain
- ต้องตรวจ compile และ fixed verification ทุกครั้ง
- ความเสี่ยง: API ผิด, assertion ไม่ตรง fixed revision หรือ compile ไม่ผ่าน
- ผลคือเปรียบเทียบ “เครื่องมือ + prompt + configuration” ไม่ใช่ตัวโมเดลเดียว ๆ

หมายเหตุ: ทั้ง 4 วิธีใช้ evaluator เดียวกัน เพื่อแยก “สร้างได้” ออกจาก “ใช้งานได้จริง”

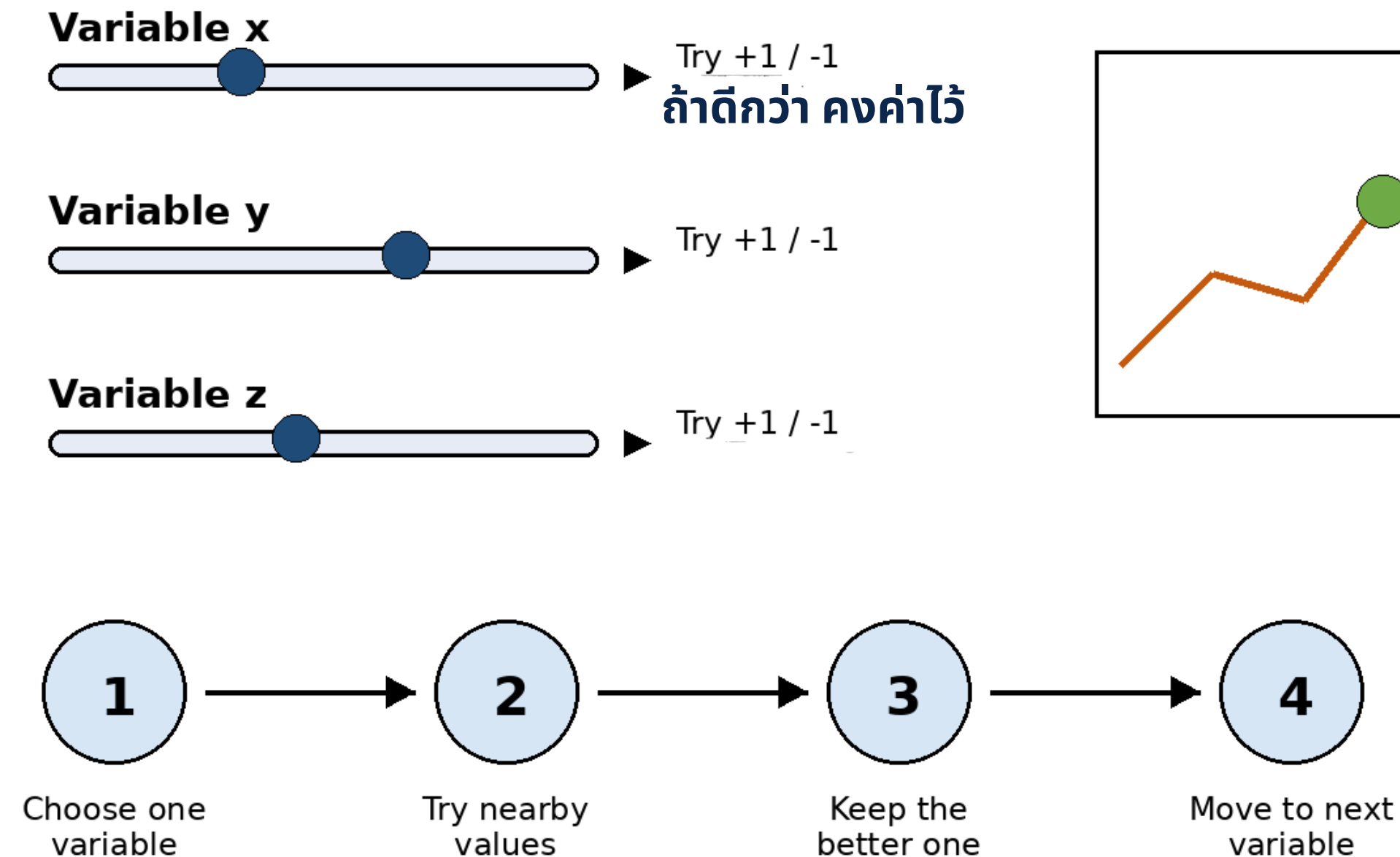


Hill climbing may stop at a better nearby point,
not always the best possible point.

แนวคิดหลัก

เริ่มจาก candidate หนึ่งชุดก่อน
ลองปรับ input หรือ setup ที่ละนิด
ถ้า fitness ดีขึ้น ก็ไปต่อจากจุดนั้น
ถ้าไม่ดีขึ้น อาจหยุดที่ local
maximum ได้

Hill Climbing มองหา “ทางที่ดีขึ้นในบริเวณใกล้เคียง” จึงอาจเจอจุดที่ดี แต่ยังไม่ใช่ว่าดีที่สุด



ต่างจาก HC ตรงไหน

ปรับทีละตัวแปรอย่างเป็นลำดับ
ลองค่าข้างเคียงก่อน
ถ้าดีขึ้น ค่อยขยับต่อในทิศเดิม
อธิบายเส้นทางการค้นหาได้ชัดเจนกว่า

AVM = ปรับตัวแปรทีละตัวอย่างเป็นระบบ เพื่อค่อย ๆ ดันคะแนน fitness ให้ดีขึ้น

หลักการคือ สร้าง test จาก fixed revision ก่อน แล้วเอา test เดิมไปรันบน buggy revision

1) Fixed revision

HC / AVM ค้นหา
input
สร้าง JUnit test
ดูพฤติกรรมที่ถูกต้อง

2) Run on fixed

Compile และ run
ถ้าผ่าน = valid test

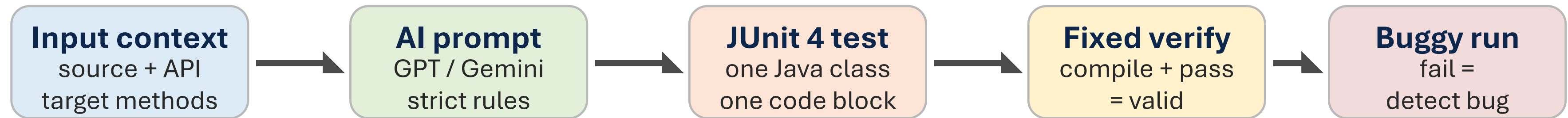
3) Same test on buggy

ใช้ source test เดิม
ไม่เปลี่ยน assertion

4) If buggy fails

ถ้า fail บน buggy
แปลว่า test จับบั๊กได้

สรุป: ถ้า test ผ่านบน fixed แต่ล้มเหลวบน buggy แปลว่า test ชุดนั้นตรวจเจอความแตกต่างที่เกิดจากบั๊ก



สิ่งที่ทั้ง 2 Prompt บังคับเหมือนกัน

- ใช้เฉพาะ fixed revision source + API
- ห้ามใช้ buggy source / patch / issue / triggering tests
- คัดค้านเป็น ONE Java class ใน ONE ``java block
- JUnit 4: org.junit.Test + org.junit.Assert
- ไม่ใส่ package declaration
- สูงสุด ONE @Test ต่อ target method
- test ต้อง deterministic และ independent
- ห้าม invent API หรือ expected behavior

Gemini Prompt

- เขียนเป็นขั้นตอนชัดเจน 4 STEP
- Analyze → Select input → Build oracle → Verify compilability
- เน้นหา branch, boundary, state change, exception และ observable output
- เลือก deterministic input ที่ meaningful

GPT Prompt

- กำหนดบทบาทเป็น expert Java unit-test engineer
- ใช้ STRICT RULES + TEST DESIGN PRIORITIES
- เน้น meaningful behavior และ strong observable assertions
- ให้ตรวจ API / setup / assertions ก่อนตอบ

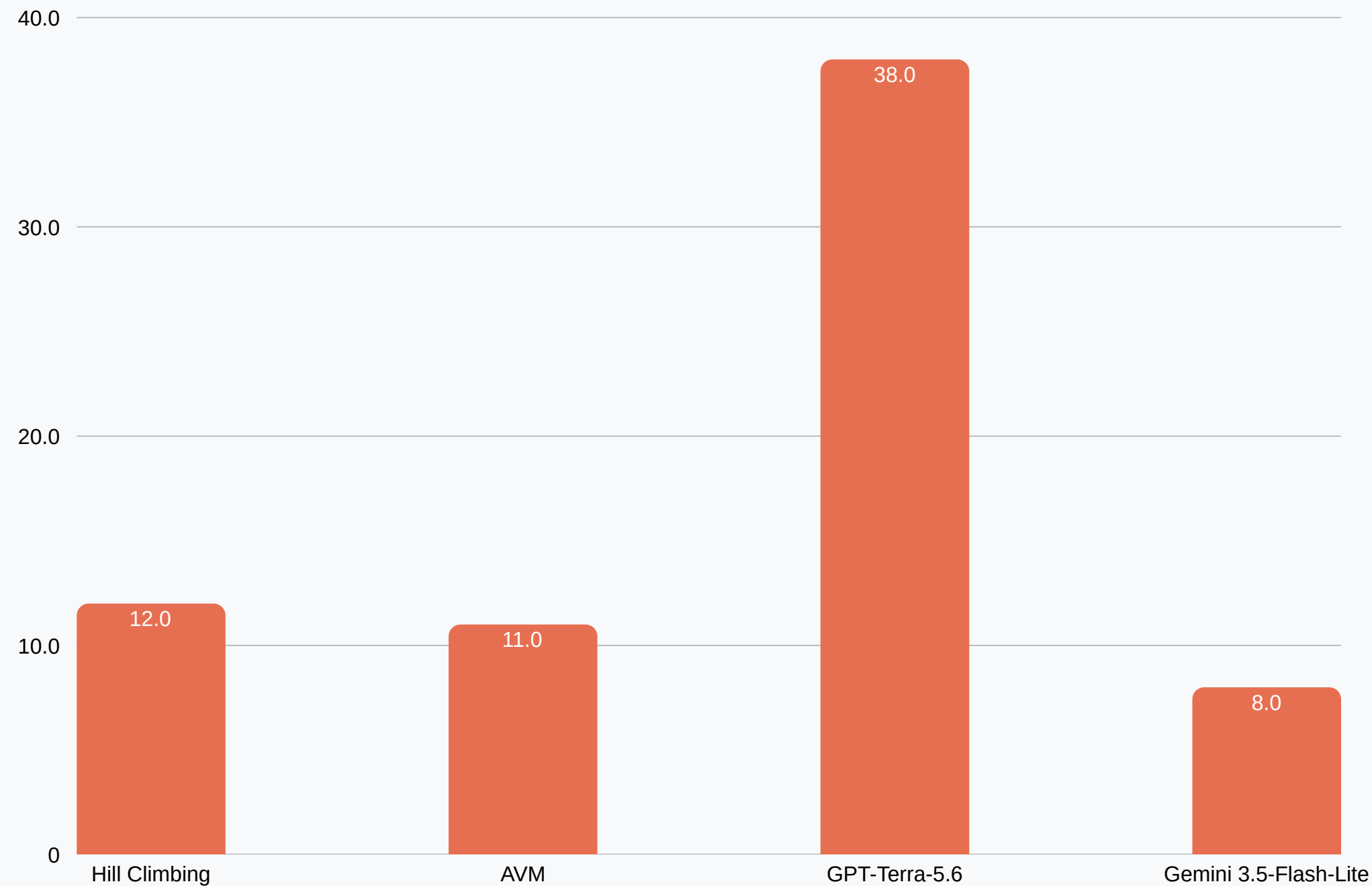
ความสามารถสร้างชุดทดสอบที่ใช้งานได้

Method	Tasks	Completed	Unsupported	Errors	Valid Suites	Validity (Valid / Completed) (%)	Valid Yield (Valid / 854) (%)
Hill Climbing	854	349	503	2	344	98.57%	40.28%
AVM	854	350	503	1	344	98.29%	40.28%
GPT	854	849	0	5	561	66.08%	65.69%
Gemini	854	853	0	1	428	50.18%	50.12%

Unsupported

Unsupported Reason	HC	AVM	Explanation
กลไกวางแผนการสร้างออบเจกต์สร้างออบเจกต์รับการเรียกเมธอดไม่ได้ภายใต้ข้อจำกัดของ public API	380	380	ระบบสร้างออบเจกต์ที่จำเป็นก่อนเริ่มทดสอบไม่ได้
ไม่มีเมธอดสาธารณะที่รับข้อมูลชนิดพื้นฐานตามที่ระบบรองรับ	94	94	ชนิดตัวแปรที่เมธอดต้องการอยู่นอกเหนือจาก primitive variable ที่ระบบตั้งค่าไว้
ไม่พบชุดทดสอบที่ใช้งานได้ในขอบเขตที่กำหนด	29	29	ไม่พบ candidate ที่ใช้งานได้ ในออบเจกต์รับการเรียกเมธอด การเตรียมสถานะ และเป้าหมายที่เลือก

การตรวจพบบั๊ก



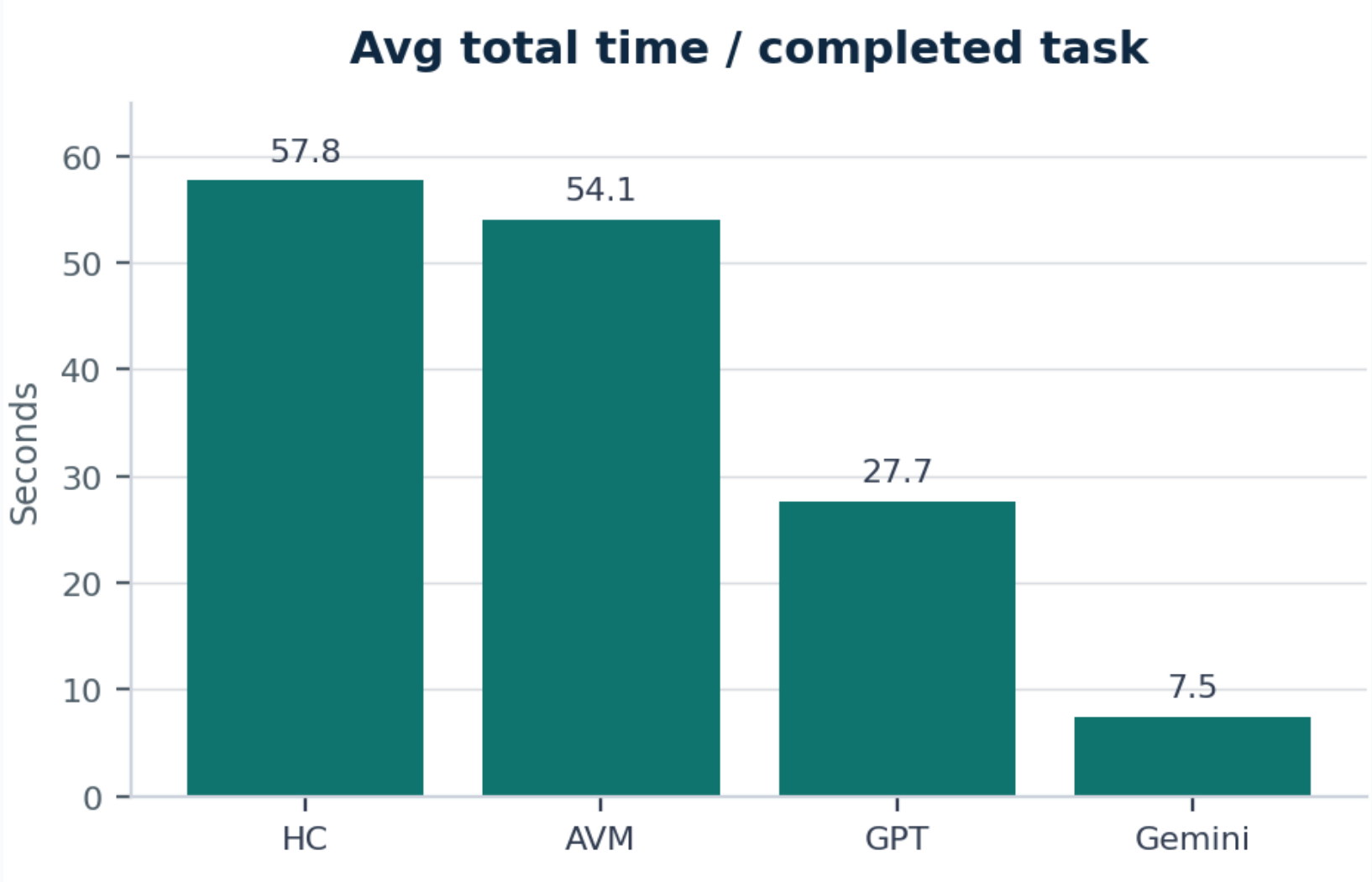
51

Unique Bug เมื่อรวม 4 วิธี

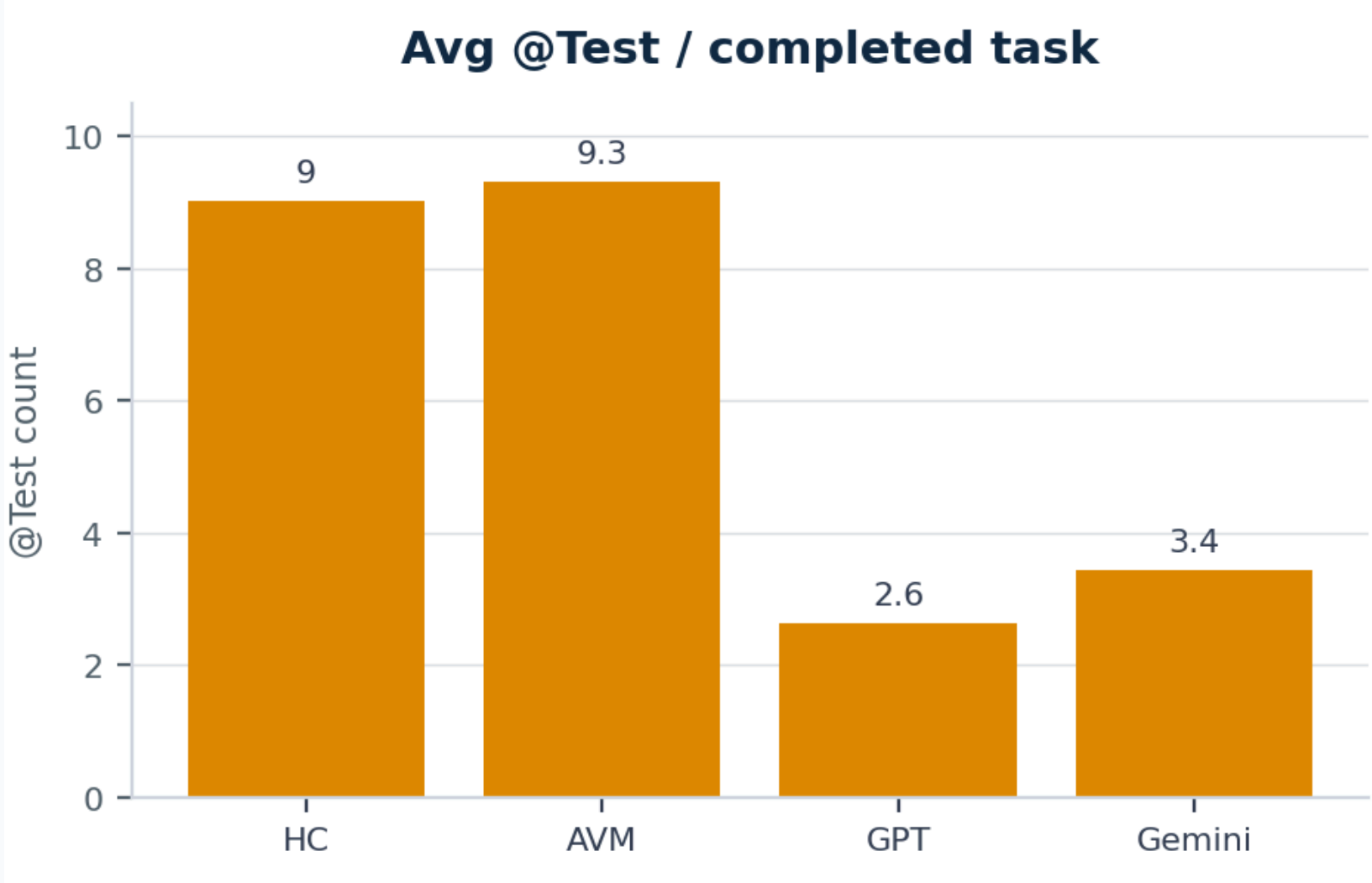
- GPT พบ 32 เคสที่วิธีอื่นไม่พบ
- Gemini พบ 5 เคสที่วิธีอื่นไม่พบ
- HC และ AVM พบตรงกัน 11 เคส

Detection rate สูงสุดที่ 4.45%
ของ 854 เคส

เวลาและจำนวนกรณีทดสอบ



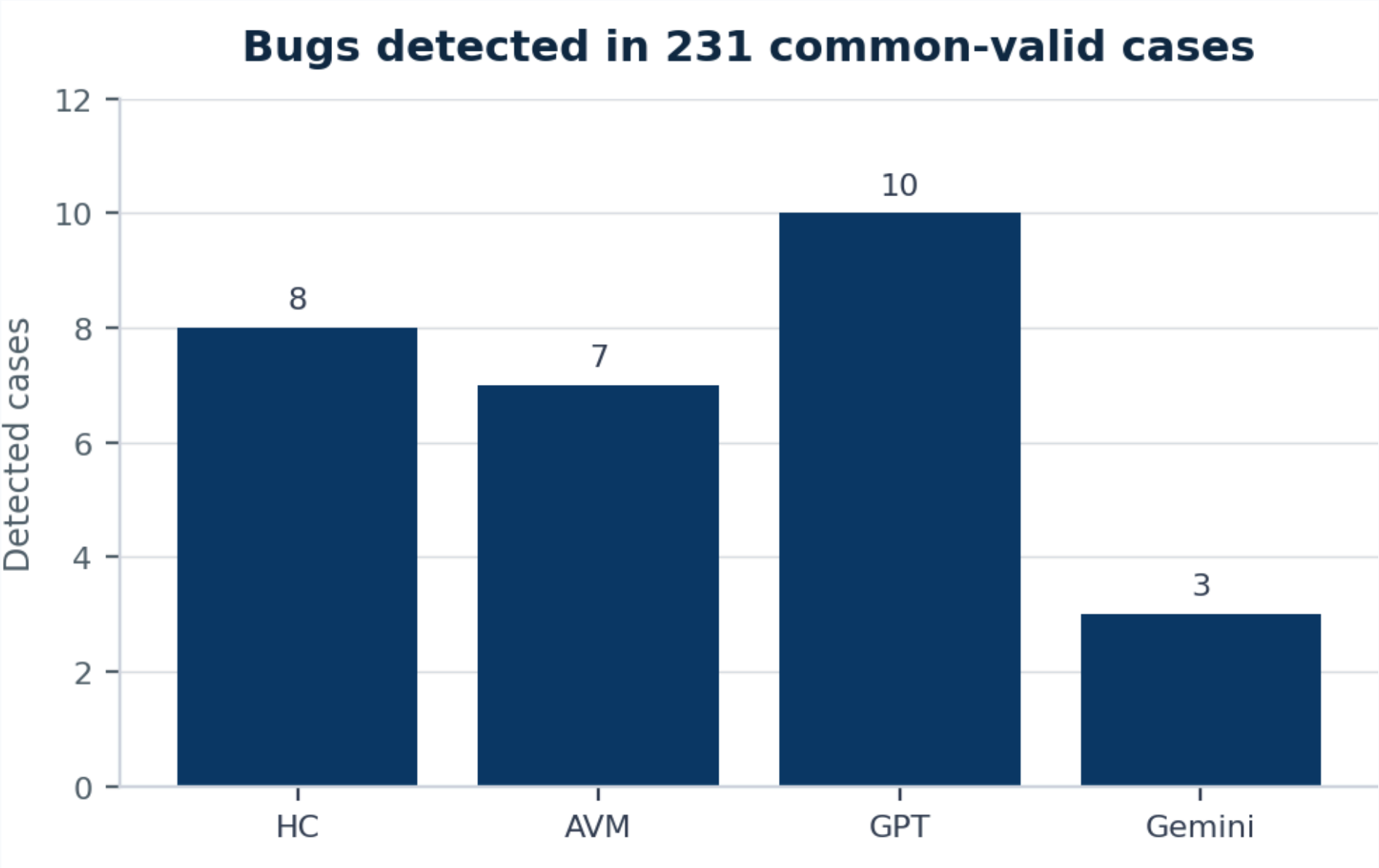
Gemini ใช้เวลาต่ำสุดในข้อมูลชุดนี้



HC/AVM สร้างหลาย test ต่อ task มากกว่า

เวลาไม่รวม checkout/compile และไม่มี hardware snapshot ครบทุกเครื่อง จึงไม่ใช่ benchmark ความเร็วทั่วไป

เปรียบเทียบบบนเคสเดียวกัน



231

เคสที่ทุกวิธี completed และ valid

เมื่อทุกวิธีถูกเทียบบบนเคสชุดเดียวกัน ความแตกต่างในการตรวจจับก็ลดลงอย่างมากต่างกันเพียง 2 เคส

ข้อควรระวัง: subset นี้คัดเฉพาะเคสที่ทุกวิธีทำได้ จึงยังมี selection bias

Coverage uu Common Valid case

Method	Bugs Detected (n, %) / 231 Cases	Instruction Coverage (%)	Branch Coverage (%)	Line Coverage (%)
Hill Climbing	8 (3.46%)	21.12%	9.93%	22.55%
AVM	7 (3.03%)	21.46%	10.25%	22.81%
GPT	10 (4.33%)	25.11%	14.26%	25.94%
Gemini	3 (1.30%)	23.53%	12.36%	24.22%

Test ที่ตรวจจับ Bug ได้ต้องมาพร้อม input/state ที่กระตุ้น fault และ oracle ที่สังเกตความต่างได้

ข้อควรระวังในการประเมินผลการทดลอง

- ใช้ seed/run เดียว จึงยังไม่เห็นความแปรปรวน
- ยังไม่มี statistical significance test
- End-of-Line sequence ต่างกันจึงส่งผลกับ Experiment ID
- ความแตกต่างของ Hardware ผู้ทดลองเวลาที่ใช้รันจึงมีผลแค่ในการทดลองนี้

ข้อสรุปและแนวทางพัฒนา

สิ่งที่ควรจำจาก benchmark นี้

GPT

Valid Yield สูงส่งผลให้มีโอกาสพบ fault ได้มากกว่า

Gemini

เวลาเฉลี่ยต่ำ แต่แลกกับ validity ที่ต่ำ

HC/AVM

Validity สูง แต่ถูกจำกัดด้วย unsupported ทำให้รองรับเคสได้น้อย

ความแตกต่างที่สังเกตได้

AI based

ยืดหยุ่นและรองรับเคสได้กว้าง
แต่ผลลัพธ์ที่สร้างออกมาอาจไม่ valid

Search based algorithm

ผลลัพธ์ค่อนข้างมั่นคงเมื่อรองรับเคส
แต่ถูกจำกัดด้วย input/object ที่ระบบสร้างได้

รอบต่อไปควรทำอะไร

- ทดลองซ้ำหลายรอบด้วยหลาย seed
- เพิ่มขั้นตอนตรวจและแก้ test ที่สร้างออกมาสำหรับ AI
- ขยายการสร้าง input และ object ให้กับฝั่ง algorithm หรือทดลองระบบผสม

**THANK
YOU**

